

● complexities with tiling:

Tile sizes not divide the matrix size and lead to low utilization:

factors affecting tile sizes:

- * Coalesced memory access
- * shared memory size / threads, warps, block size alignment.
- * Divisibility of matrix dim / divisible matrix dim.

memory alignment:

memory comes in bursts
loading tiles are fast if burst align with matrix

Verilog has a major impact through alignment

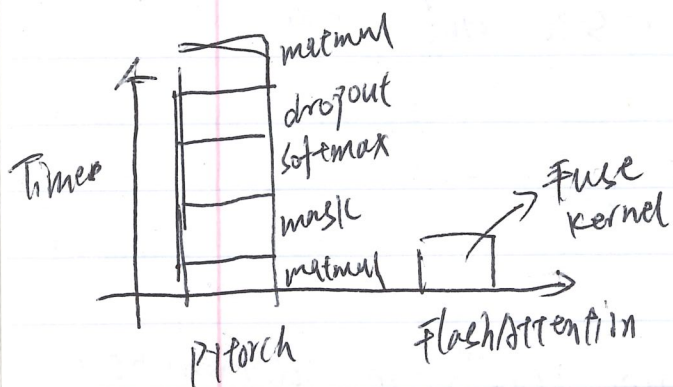
Summary of part #2: making ML loads go faster.

(#1) $\Rightarrow$ Reduce memory access { coalescing
Fusion

(#2) $\Rightarrow$ Move global memory to shared memory: Tiling

(#3) $\Rightarrow$ Trade memory for compute/efficiency: { a quantization
Recomputation

(part #3): using what we know to understand Flash Attention



Attention	Standard	Flash Attention
Gflops	16.6	75.2
HBM R/W	40.3	4.4
Runtime (ms)	41.7	7.3

K/Q/V: three learnable Matrix for Attention computation

⇓

Flash Attention: ^{memory} tiling for K/Q/V matrix multiply

⇓ what do we do for softmax?

⇒ incremental computation of the softmax

normal softmax ⇒ online softmax (keep track, update, telescoping)

⇓

compute softmax tile by tile

From Dao 2023, we see:

1): tile-wise computation of inner products

2): Fusion of exponential operator

3): tile-wise online softmax computation, telescoping sum trick